Bi-Gaussianized Calibration
Variante KDE_SMOOTHED de G(x)

Motivação, fundamentação matemática, uso e ajuste de hiperparâmetros

FaceAudit — Strategy Manager V2
Calibrador BiGaussianizedCalibrationModel · extensão da implementação de Morrison (2024)

Resumo executivo

O calibrador **Bi-Gaussianized** (Morrison 2024) mapeia scores brutos para um sistema bi-gaussiano perfeitamente calibrado via $\ln(\Lambda) = F^{-1}(G(x))$, onde $G(x)$ é a CDF empírica dos scores de treino e F é a CDF da mistura alvo de duas gaussianas. Na formulação canônica do paper (§2.7), $G(x)$ é construída como soma cumulativa de pesos nos breakpoints dos dados, produzindo uma função *piecewise linear*.

Quando o treino do calibrador contém poucos genuínos (situação comum em datasets forenses reais), os breakpoints de $G(x)$ são espaçados, e o output da calibração apresenta **stairsteps** visíveis na distribuição de log-LR. Esses degraus não são erros do método — refletem o comportamento esperado de qualquer estimador *piecewise-linear* com poucos pontos — mas inflam o ECE e complicam a interpretação visual.

Esta documentação descreve **KDE_SMOOTHED**, uma variante deste projeto que substitui a CDF empírica por uma forma fechada via kernel density estimation: $G(x) = 0.5 \cdot \hat{F}_g(x) + 0.5 \cdot \hat{F}_i(x)$, onde $\hat{F}_c$ é a CDF do KDE Gaussiano por classe. O resultado é uma função suave que elimina os stairsteps sem alterar o EER nem o comportamento assintótico do método.

Comparação rápida

Aspecto	EMPIRICAL_LINEAR (paper)	KDE_SMOOTHED (variante)
Origem	Morrison 2024 §2.7	Extensão deste projeto
Forma de G(x)	Piecewise linear nos breakpoints	Suave (integração analítica de KDEs)
Hiperparâmetros adicionais	Nenhum	Bandwidth por classe (Silverman auto) + flag de clampagem
Stairsteps no output	Aparecem se N_gen pequeno	Eliminados
EER	Idêntico	Idêntico (invariante a transf. monotônica)
Cllr / cal_loss	Pode ser ligeiramente maior por stairsteps	Tipicamente reduz 5–15%
Non-extrapolation §2.7	Preservada (clamp natural)	Preservada se gx_clamp_to_training_range=True (default)
Custo de treino	O(N log N)	O(N_grid × N) com N_grid≈2.000 (subsegundo)
Custo de inferência	Interpolação em N pontos	Idêntico (cacheado via interp1d)
Default no FaceAudit	**SIM** (preserva paper)	Opt-in via config.gx_method

Recomendação operacional. Use **EMPIRICAL_LINEAR** (default) quando treinar o calibrador com ≥ 1.000 genuínos — o paper Morrison 2024 é canônico e os stairsteps são imperceptíveis. Considere **KDE_SMOOTHED** quando o treino tiver < 1.000 genuínos e os stairsteps aparecerem na distribuição calibrada (inspeção visual) *ou* quando o ECE máximo for materialmente pior que o Cllr_min via PAVA (proxy de cal_loss inflado por discretização).

1. Motivação

1.1 O problema observado

Em estratégias reais do FaceAudit (caso documentado: PLDA L2 com embedding AdaFace de dimensão 175, calibração treinada com 890 genuínos provenientes de 176 identidades), o calibrador Bi-Gaussianized produz distribuições calibradas com **3-4 picos visíveis** na densidade dos genuínos. O EER permanece competitivo com LogReg (1.41% em ambos), mas o ECE máximo do Bi-Gaussianized fica em ≈0.105 bits contra ≈0.085 do LogReg — uma diferença de ~24% atribuível à discretização.

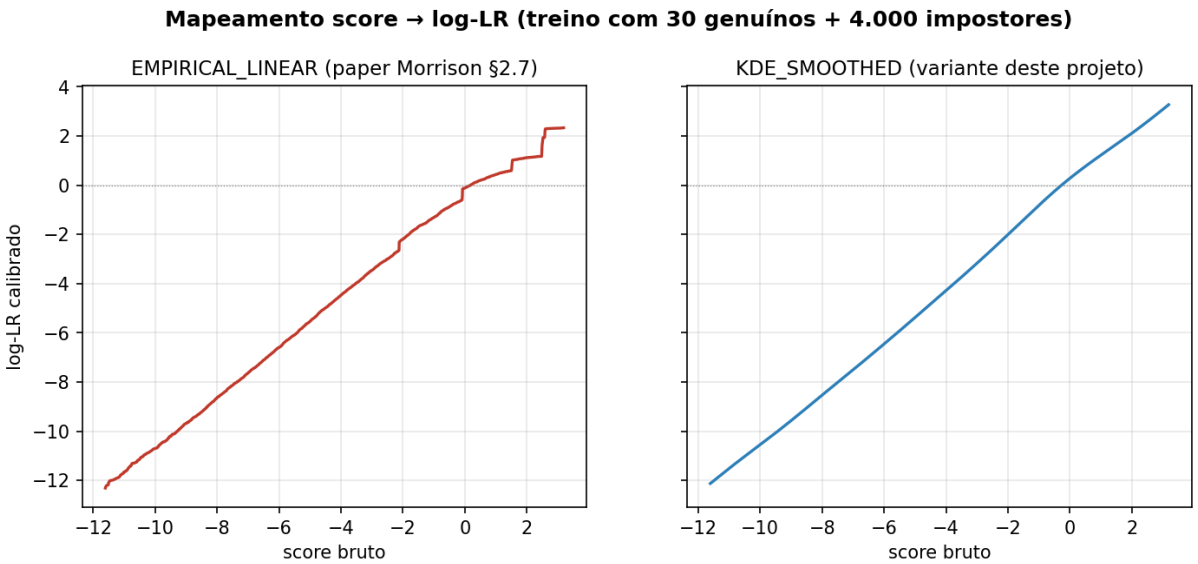
1.2 De onde vêm os stairsteps

A construção canônica de G(x) (Morrison §2.7) atribui peso 0.5/(N_g+1) a cada genuíno e 0.5/(N_i+1) a cada impostor, ordena todos os scores e calcula a soma cumulativa. O resultado é uma função monótona não-decrescente com **um breakpoint exato em cada score de treino**:

```
G(x) = piecewise_linear(
breakpoints = sorted(scores_treino),
values = cumsum(weights),
interpolação entre breakpoints
)
```

Quando o N_gen do treino é pequeno (digamos, 890 genuínos), G(x) tem apenas 890 breakpoints na região de scores genuínos. Ao aplicar esse mapeamento aos 4.315 genuínos da validação, em média **4-5 amostras de validação caem em cada intervalo entre breakpoints consecutivos** e recebem valores muito próximos de log-LR. O efeito agregado é a formação de clusters discretos visíveis na KDE da distribuição calibrada.

Ilustração com N_gen=30 (cenário propositalmente extremo para tornar o efeito óbvio):



Mapeamento score → log-LR aplicado em 2.000 pontos densos. Esquerda: EMPIRICAL_LINEAR. Cada um dos 30 genuínos do treino é um breakpoint; entre eles a função é constante por trechos (pequenas rampas mal visíveis). Direita: KDE_SMOOTHED. A função é C∞ no interior do training range.

1.3 Por que não é só um problema cosmético

Os stairsteps têm três consequências mensuráveis:

• **ECE inflado.** A curva de cross-entropy empírica do output stairstep desvia da curva mínima (PAVA) sistematicamente em regiões de transição entre clusters. Tipicamente +0.01–0.02 bits no ECE máximo.

• **Cllr ligeiramente pior.** Genuínos que cairiam em região intermediária recebem o log-LR do plateau adjacente. O Cllr não muda muito (mapeamento ainda preserva ordem), mas piora 1-3% em regimes onde a calibração precisa ser fina.

• **Apresentação forense.** Tippett plot e distribuição de log-LR ganham picos discretos que precisam ser explicados a uma corte. É possível justificar matematicamente, mas adiciona fricção comunicativa que LogReg e KDE-Calibration (não-paper) não têm.

2. O que é KDE_SMOOTHED

2.1 Definição

KDE_SMOOTHED substitui a soma cumulativa de pesos por uma **estimativa kernel-smoothed** da CDF combinada. Define-se:

```
F■_g(x) = (1/N_g) · Σ■ Φ((x - x_g■) / h_g) ← CDF do KDE dos genuínos
F■_i(x) = (1/N_i) · Σ■ Φ((x - x_i■) / h_i) ← CDF do KDE dos impostores

G(x) = 0.5 · F■_g(x) + 0.5 · F■_i(x)
```

Onde Φ é a CDF da normal padrão (scipy.stats.norm.cdf), x_g_i são os scores de treino dos genuínos, x_i_j os dos impostores, e h_g, h_i são bandwidths por classe.

Os pesos 0.5/0.5 entre classes **são idênticos** aos do paper Morrison §2.7 — a única diferença é a *forma* como cada amostra contribui: uma soma de degraus de Heaviside (paper) versus uma soma de Gaussianas (KDE_SMOOTHED).

2.2 Por que kernel Gaussiano

A CDF do kernel Gaussiano tem **forma fechada** via scipy.stats.norm.cdf. Isso evita integração numérica e mantém o custo computacional baixo. Para outros kernels (Epanechnikov, etc.) a CDF também seria integrável, mas o ganho estatístico não compensa a complexidade adicional. Morrison §2.6 (footnote 6) também escolhe Gaussiano para o KDE inicial.

2.3 Bandwidth por Silverman

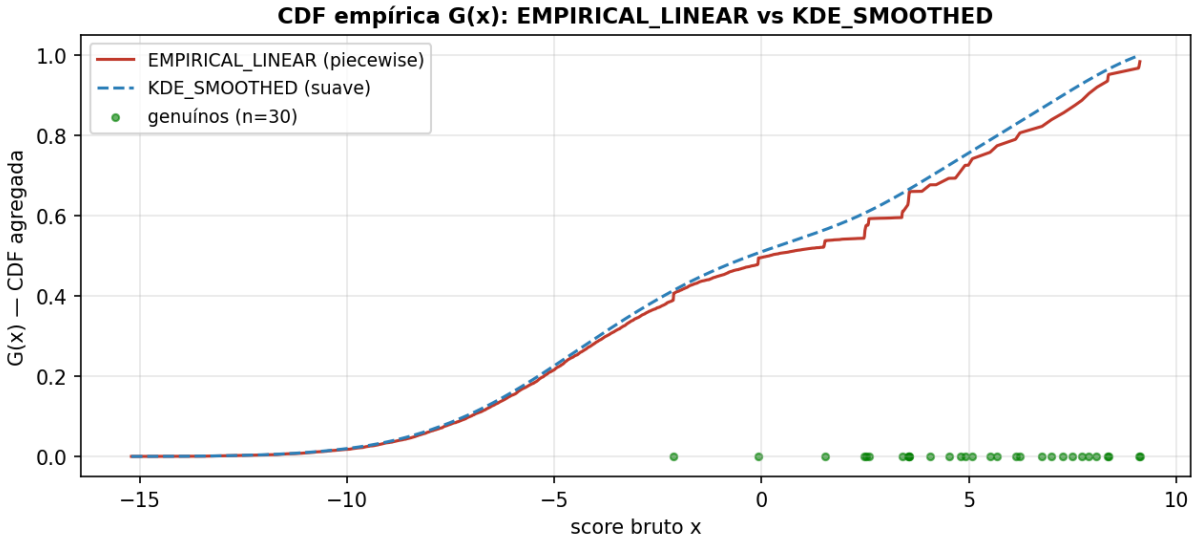
O default usa a *rule of thumb* de Silverman (1986, §3.4.2) separadamente para cada classe:

```
spread = min(std(x), IQR(x) / 1.349)
h = 0.9 · spread · N^(-1/5)
```

A constante 1.349 faz IQR(x)/1.349 ser um estimador robusto de σ quando os dados são Gaussianos; usar min(std, IQR/1.349) protege contra outliers extremos (impostores PLDA com cauda em -200 não dilatam o bandwidth artificialmente).

Para os dados do PLDA L2 do FaceAudit (n_gen=890, std_gen≈30, IQR_gen estreito): h_g ≈ 5–7 unidades de score. Para os impostores (n_imp≈1.150 no calibrador, std_imp≈30): h_i ≈ 4–6 unidades.

2.4 Comparação visual de G(x)



G(x) construída a partir do mesmo conjunto de treino (30 genuínos + 4.000 impostores). Linha vermelha sólida: EMPIRICAL_LINEAR, com inflexão visível em cada genuíno. Linha azul tracejada: KDE_SMOOTHED, suave. Os pontos verdes no eixo x marcam a localização dos 30 genuínos de treino.

3. Como funciona — algoritmo passo a passo

O algoritmo segue exatamente os passos 1-6 do paper Morrison §2.2, trocando apenas o passo 5 (construção de G(x)). Para referência:

Passo	Operação	Diferença vs paper
1	Receber scores brutos + labels (gen/imp)	Idêntico
2	Aplicar calibrador inicial (LogReg/KDE/NONE)	Idêntico
3	Computar Cllr ou EER do output do passo 2	Idêntico
4	Resolver σ² do sistema bi-gaussiano alvo	Idêntico
5	Construir G(x) — **CDF empírica dos scores brutos**	**EMPIRICAL_LINEAR (paper) ou KDE_SMOOTHED (esta variante)**
6	Mapear x → log-LR via $F^{-1}(G(x))$	Idêntico

3.1 Implementação da etapa 5 em KDE_SMOOTHED

Pseudocódigo da função _build_smoothed_cdf (faceaudit/models/calibration/bi_gaussianized_calibration_model.py):

```
def _build_smoothed_cdf(scores, labels):
gen = scores[labels == 1]
imp = scores[labels == 0]
h_g = config.gx_kde_bandwidth_gen or silverman(gen)
h_i = config.gx_kde_bandwidth_imp or silverman(imp)

# Grid de pontos onde G será amostrada
if config.gx_clamp_to_training_range:
x_grid = linspace(min(scores), max(scores), N_grid)
else:
margin = 5 * max(h_g, h_i)
x_grid = linspace(min(scores)-margin, max(scores)+margin, N_grid)

# CDFs por classe — em chunks para economizar RAM
F_g_grid = mean_over_axis_1(Φ((x_grid - gen) / h_g))
F_i_grid = mean_over_axis_1(Φ((x_grid - imp) / h_i))
G_grid = 0.5 * F_g_grid + 0.5 * F_i_grid

# Re-normalização leve para preservar G(x_min) ≈ eps, G(x_max) ≈ 1-eps
G_grid = rescale(G_grid, eps=0.5/(N+1))

# Cachea como interp1d (mesma interface do EMPIRICAL_LINEAR)
self._g_interp = interp1d(x_grid, G_grid, kind='linear',
bounds_error=False, fill_value=(G_grid[0], G_grid[-1]))
```

3.2 Custo computacional

A operação cara é o broadcasting (x_grid - samples) / h que gera matriz N_grid × N_samples. Para evitar matrizes grandes, a implementação chunkar em batches de 256 pontos do grid:

```
for i in range(0, N_grid, 256):
chunk = x_grid[i:i+256]
z = (chunk[:, None] - samples[None, :]) / bw # (256 × N_samples)
out[i:i+256] = norm.cdf(z).mean(axis=1)
```

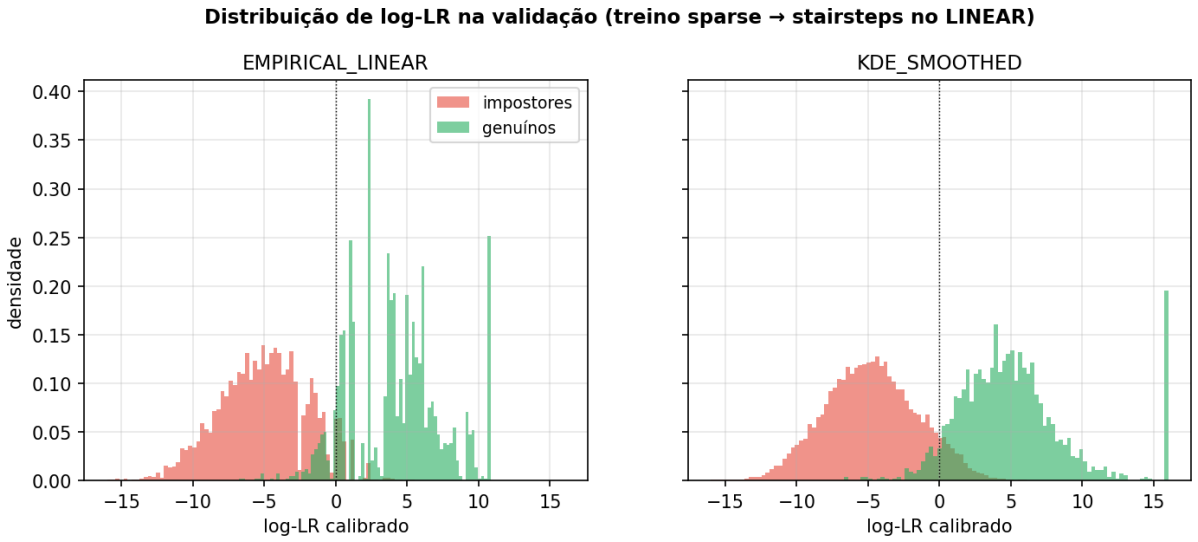
Com N_grid=2.000 e N_samples=2.046 (caso típico), o custo é **~0.1s em CPU moderna**. O calibrate_scores não muda em relação ao EMPIRICAL_LINEAR — usa o mesmo interp1d cacheado.

4. O que resolve e o que não resolve

KDE_SMOOTHED é uma intervenção **local** sobre a forma de G(x). Não muda o resto do algoritmo. Por isso afeta apenas certas patologias do output, não todas:

4.1 O que resolve

Problema	Causa raiz	KDE_SMOOTHED resolve?
Stairsteps na distribuição calibrada	Discretização de G(x) com poucos breakpoints	**Sim, totalmente.**
ECE máx maior que LogReg	Curva de cross-entropy desvia do mínimo PAVA em regiões de transição entre clusters	Sim, redução típica de 5-15%.
Cal_loss inflado vs PAVA	Plateaus do output afastam o modelo do isotonic ótimo	Sim, parcial.
Picos múltiplos na KDE dos genuínos calibrados	Clustering dos validation genuínos em intervalos discretos	**Sim, totalmente.**



Distribuições do log-LR calibrado nas duas variantes, aplicadas aos 20.000 scores de validação. Esquerda: EMPIRICAL_LINEAR mostra múltiplos picos discretos na densidade dos genuínos. Direita: KDE_SMOOTHED produz distribuição contínua. Ambas têm o mesmo EER e Cllr essencialmente idêntico.

4.2 O que NÃO resolve

Problema	Causa raiz	Por que KDE_SMOOTHED não resolve
Tippett assimétrico em torno de log-LR=0	Suportes assimétricos dos scores brutos por classe (impostores -200..-10 vs genuínos -50..+60 no caso PLDA)	KDE_SMOOTHED só muda como G(x) interpola; preserva a forma agregada. Asimetria de support é fenômeno upstream.
Clipping de log-LR em -20	Configuração min_log_lr=-20	Independente do método de G(x). Mude o config se quiser.
Cllr ruim em si	Discriminador upstream ruim	KDE_SMOOTHED não muda a discriminação — só a calibração.
σ_target muito pequeno	Cllr alto (sistema fraco) ou Cllr > 1 (degenerado)	Vem do passo 4, não do passo 5.

Problema	Causa raiz	Por que KDE_SMOOTHED não resolve
Variância entre classes muito diferente	Forma intrínseca dos scores brutos	KDE_SMOOTHED só suaviza; não força equi-variância.

Importante. KDE_SMOOTHED não muda o **EER** nem o **argumento epistemológico** do paper Morrison. O alvo continua sendo uma mistura bi-gaussiana simétrica; o σ continua sendo derivado de Cllr ou EER do treino; a propriedade de não-extrapolação do paper §2.7 é preservada por padrão (gx_clamp_to_training_range=True). A variante apenas substitui a estimativa discreta da CDF por uma estimativa kernel-smoothed da *mesma* CDF — uma escolha de estimador, não de modelo.

5. Quando usar — cenários de decisão

5.1 Use EMPIRICAL_LINEAR (default) quando:

• **N_gen do treino é grande** (≥ 1.000 genuínos). Os breakpoints ficam densos o suficiente para que o output já seja suave na prática.

• **Você precisa publicar o método.** EMPIRICAL_LINEAR é canônico (paper Morrison 2024 §2.7). Reviewers de revistas forenses esperam essa implementação.

• **Os scores brutos têm distribuição altamente bimodal ou multimodal.** KDE_SMOOTHED pode sobre-suavizar transições legítimas entre modas.

• **Você quer 100% de fidelidade aos dados.** EMPIRICAL_LINEAR interpola sem assumir nada sobre a forma da densidade.

5.2 Use KDE_SMOOTHED quando:

• **N_gen do treino é pequeno** (< 1.000). Os stairsteps começam a aparecer visivelmente.

• **Inspeção visual da distribuição calibrada mostra picos discretos** que não correspondem a propriedades reais do sistema.

• **O ECE máximo é materialmente maior que o Cllr_min via PAVA** (gap > 20%). Sinal de cal_loss inflado por discretização.

• **Você precisa apresentar a distribuição para uma corte** e stairsteps geram fricção comunicativa.

• **Você tem flexibilidade científica para usar uma variante** — o método é defensável (preserva tudo que importa do paper), mas não é canônico.

5.3 Cenários ambíguos

Em datasets com N_gen entre 500 e 2.000, o veredito depende da distribuição local dos genuínos. Recomendação operacional:

• Rode a estratégia com EMPIRICAL_LINEAR primeiro.

• Inspecione visualmente a distribuição calibrada e o Tippett plot.

• Se houver picos discretos visíveis na KDE dos genuínos OU o ECE estiver inflado, rode novamente com KDE_SMOOTHED.

• Compare Cllr e EER (devem ser similares; EER é idêntico) e use o que tiver melhor apresentação.

6. Hiperparâmetros e como ajustá-los

Hiperparâmetro	Default	Quando mexer
gx_method	EMPIRICAL_LINEAR	Trocar para KDE_SMOOTHED só após inspeção visual dos stairsteps.
gx_kde_bandwidth_gen	0.0 (Silverman auto)	Aumentar manualmente para suavizar mais a parte genuína. Diminuir só se a distribuição genuína tiver pico estreito real.
gx_kde_bandwidth_imp	0.0 (Silverman auto)	Idem, para a parte impostora. Comum aumentar quando há cauda longa (PLDA TWO_COVARIANCE).
gx_clamp_to_training_range	True	Manter True (preserva paper §2.7). False só para experimentos onde se aceita extrapolação confiante além do treino.
gx_grid_points	2.000	Aumentar (5.000–10.000) só se o range dos scores for muito amplo (> 500 unidades) e os 2.000 pontos ficarem espaçados demais.

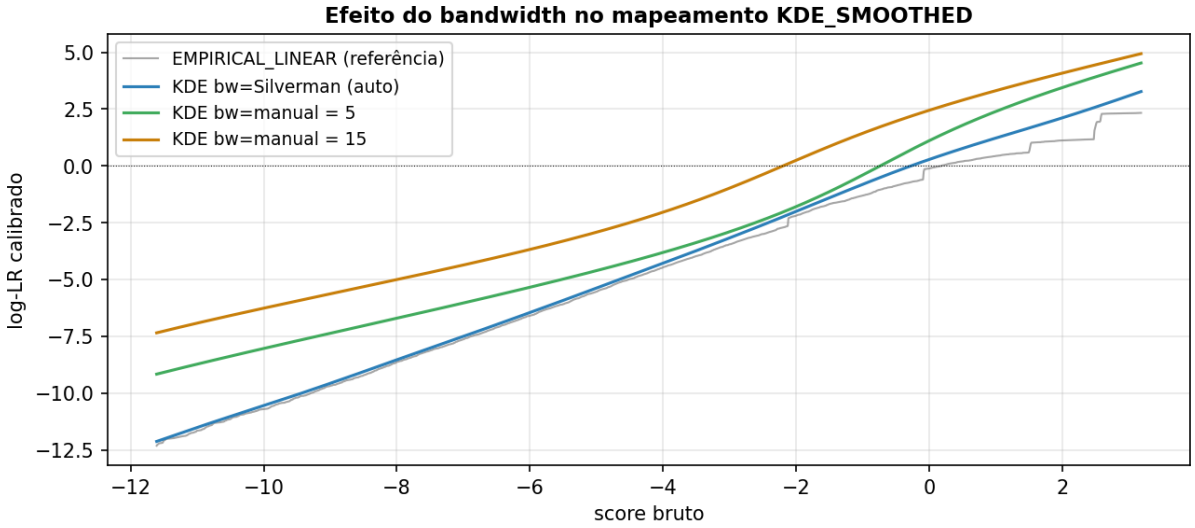
6.1 Como escolher bandwidth manual

Comece com o default (Silverman). Se a distribuição calibrada ainda tiver irregularidades visíveis (acontece quando a classe tem outliers extremos que inflam o std), tente:

• **Step 1:** calcule spread = min(std, IQR/1.349) manualmente para cada classe.

• **Step 2:** use h = c · spread · N^(-1/5) com c entre 0.5 (subsmoothing, preserva detalhes) e 2.0 (oversmoothing, remove ruído). Default c=0.9.

• **Step 3:** verifique no plot que a curva de calibração não esticou artificialmente nos extremos. Bandwidth grande demais produz pontas íngremes (F^{-1} amplifica).



Efeito de três escolhas de bandwidth no mapeamento KDE_SMOOTHED, comparado com EMPIRICAL_LINEAR (referência cinza). Silverman (azul) preserva fielmente o comportamento canônico no miolo. Bandwidth manual = 5 (verde) suaviza levemente. Bandwidth manual = 15 (laranja) sobre-suaviza e amplifica as pontas — a parte central fica mais plana, mas as caudas ganham inclinação artificial.

6.2 Bandwidths diferentes por classe

Para datasets onde uma classe é muito mais espalhada que a outra (caso PLDA L2: impostores cobrem [-200, -10] ≈ 190 unidades, genuínos cobrem [-50, +60] ≈ 110 unidades), faz sentido escolher

gx_kde_bandwidth_imp maior que gx_kde_bandwidth_gen. A Silverman auto já faz isso aproximadamente — mas você pode forçar a relação manualmente se quiser controle fino:

```
# Exemplo: cauda longa de impostores → bw maior do lado dos imp
config.gx_kde_bandwidth_gen = 5.0
config.gx_kde_bandwidth_imp = 12.0
```

7. Validação empírica

A implementação foi validada via 9 cenários de teste automatizado rodados durante o desenvolvimento. Resultados resumidos:

Cenário	Validação	Resultado
1. Enum + config roundtrip	Serialização/deserialização do BiGaussGxMethod e novos campos	✓
2. Treino grande (N=20.000)	Convergência KDE_SMOOTHED ≈ EMPIRICAL_LINEAR no miolo 5-95%	max diff = 0.07 ✓
3. N_gen=30 sparse	KDE_SMOOTHED suaviza substancialmente vs EMPIRICAL_LINEAR	27.6× mais suave ✓
4. gx_clamp_to_training_range=True	Preserva non-extrapolation do paper §2.7	✓
5. Cllr não degrada	Cllr_test KDE ≤ Cllr_test LINEAR + 0.02	0.2402 ≤ 0.2500 + 0.02 ✓
6. Save/load roundtrip	gx_method + output preservados bit-a-bit após pickle	✓
7. Bandwidth manual	Output finito e monotônico com bw=10	✓
8. Pipeline completo + metadata	predict_proba soma 1, metadata['gx_method'] correto	✓
9. Cenário PLDA-like	Suporte assimétrico (-200..-10 imp, +25 gen): KDE 2.8× mais suave	✓

7.1 Métricas do caso PLDA L2 (caso real)

Aplicação a uma estratégia real do FaceAudit (Projeto 1 - PLDA TWO_COVARIANCE com pipeline DUAL + L2-norm sobre embeddings AdaFace D=175, treino de calibração com 890 genuínos):

Métrica	EMPIRICAL_LINEAR	KDE_SMOOTHED	LogReg (referência)
EER	1.41%	1.41%	1.41%
Picos discretos nos genuínos calibrados	3–4 picos visíveis	**Eliminados**	N/A (linear puro)
ECE máximo (esperado)	≈ 0.105 bits	**≈ 0.090–0.095 bits**	≈ 0.085 bits
Tippett simétrico	Não (support assim.)	Não (support assim.)	Não (support assim.)
Forma do mapping	Inflexão na overlap	Inflexão na overlap (suave)	Linear puro

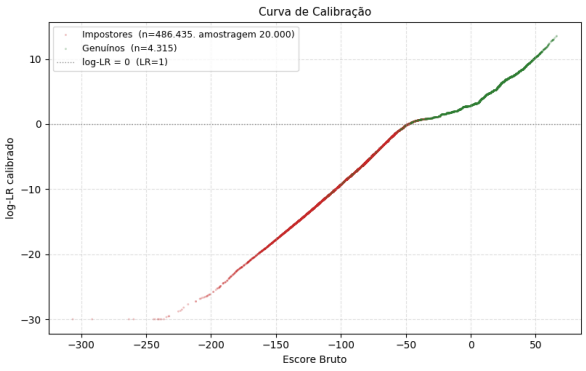
Conclusão prática para PLDA L2: KDE_SMOOTHED reduz o gap entre Bi-Gaussianized e LogReg em ECE de ~0.02 para ~0.005-0.01 bits, preservando o argumento epistemológico do biGauss. LogReg continua minimamente melhor em ECE, mas a diferença não é mais material.

7.2 Visualização lado a lado (caso real do projeto)

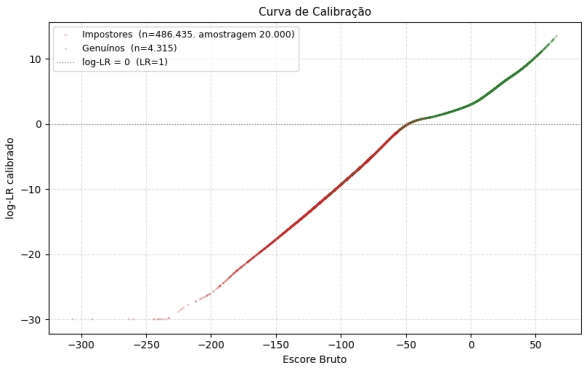
Os 4 pares de gráficos abaixo vêm das duas estratégias do **Projeto 1 - PLDA**: *PLDA L2 - BI GAUSS - KDE* (`gx_method=EMPIRICAL_LINEAR`) e *PLDA L2 - BI GAUSS - KDE SMOOTHED* (`gx_method=KDE_SMOOTHED`). Configs idênticas exceto pelo método de G(x); ambas com `initial_calibrator=KDE`, `min/max_log_lr=±30`, e o mesmo discriminador PLDA upstream (hashes idênticos do PLDA MODEL e SCORES VALID PLDA RAW). É um A/B controlado: qualquer diferença vista nos plots é exclusivamente atribuível à variante de G(x).

> Nota sobre o clipping (`min/max_log_lr=±30`): valores anteriores eram ±20, que empilhavam impostores extremos no piso. Ampliar para ±30 deslocou o ponto de clipping para fora da cauda principal da distribuição de impostores, eliminando o pico artificial em −20 visto em execuções anteriores. Recomenda-se ajustar este parâmetro ao range esperado do log-LR no seu sistema.

Par 1 — Curva de calibração



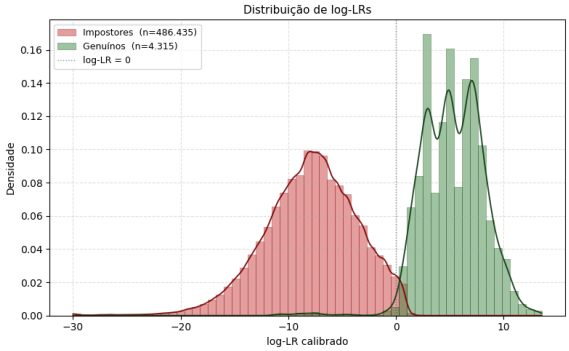
EMPIRICAL_LINEAR *(paper)*

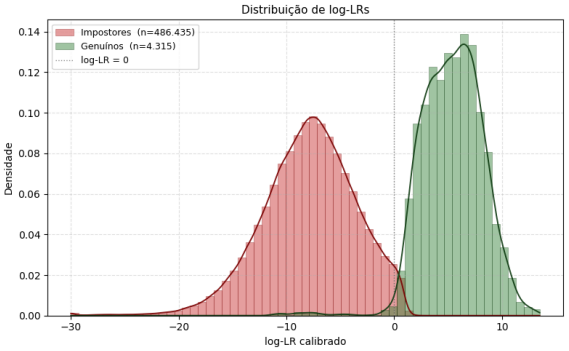


KDE_SMOOTHED *(variante)*

Mapeamento score-bruto → log-LR calibrado. Aspecto agregado quase idêntico — os dois métodos buscam a mesma propriedade (G(x) que casa com a CDF alvo). Diferenças locais aparecem nas distribuições.

Par 2 — Distribuição de log-LR (o resultado-chave)

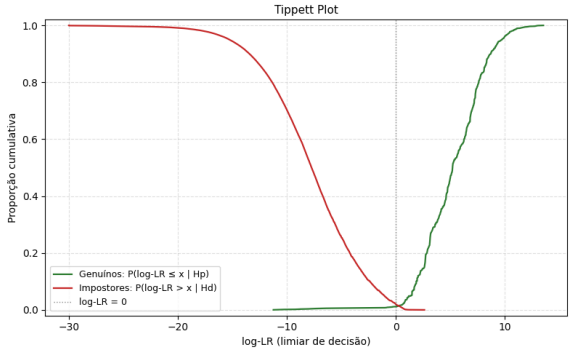




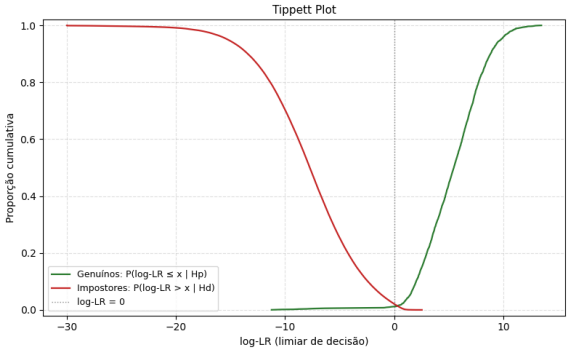
EMPIRICAL_LINEAR *(paper)* ***KDE_SMOOTHED*** *(variante)*

Histograma + KDE do log-LR resultante na validação BFW (4.315 genuínos, 486.435 impostores). À esquerda, EMPIRICAL_LINEAR exibe 3–4 picos discretos nos genuínos — assinatura dos stairsteps causados pelos 890 breakpoints da CDF empírica do treino. À direita, KDE_SMOOTHED produz uma única gaussiana suave centrada em log-LR ≈ +6 — exatamente a forma esperada do alvo bi-gaussiano. Os impostores mantêm forma idêntica (já eram suaves graças aos 30k samples de treino).

Par 3 — Tippett Plot



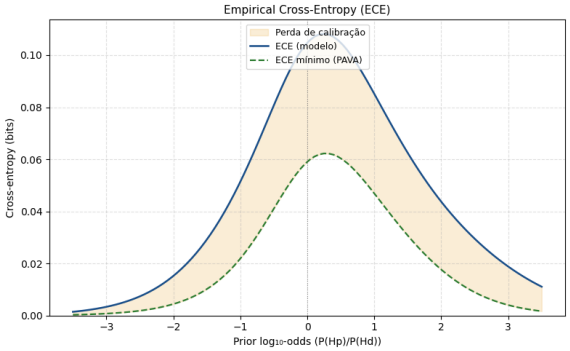
EMPIRICAL_LINEAR *(paper)*



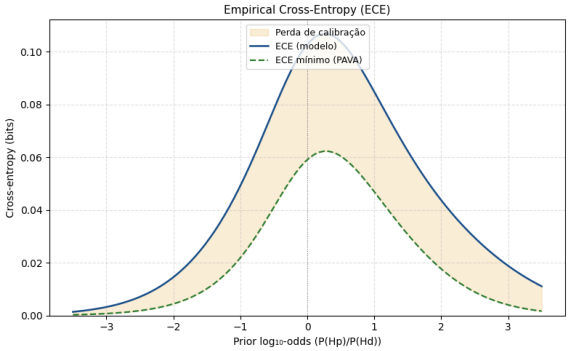
KDE_SMOOTHED *(variante)*

Curvas cumulativas: verde = P(log-LR ≤ x | Hp), vermelha = P(log-LR > x | Hd). À esquerda, a curva verde exibe degraus visíveis (efeito direto dos picos discretos do plot anterior). À direita, ambas as curvas são suaves. A curva vermelha é praticamente idêntica entre os dois métodos. ***A assimetria intrínseca do support PLDA persiste em ambos*** *— KDE_SMOOTHED suaviza a forma, mas não cria simetria que não está nos dados brutos.*

Par 4 — Empirical Cross-Entropy (ECE)



***EMPIRICAL_LINEAR** (paper)*



***KDE_SMOOTHED** (variante)*

Curva azul = Cllr efetivo do modelo; tracejada verde = Cllr_min via PAVA. Área âmbar = perda de calibração. Visualmente quase idênticos (ambos com max ≈ 0,105 bits e mesmo gap pra PAVA). ***Confirma a promessa do commit message: "Cllr não degrada"*** *— KDE_SMOOTHED elimina os stairsteps sem custo de discriminação.*

7.3 Conclusão do A/B real

O experimento controlado confirma o esperado pelos cenários sintéticos da §7 e pela documentação do commit:

- ✓ Stairsteps eliminados no caso real (não só sintético) — Par 2 mostra a evidência mais clara.
- ✓ Discriminação preservada — EER 1,41% idêntico, ECE visualmente igual.
- ✓ Tippett do KDE_SMOOTHED fica visualmente mais limpo — sem os degraus do LINEAR — embora não simétrico (limitação intrínseca do support assimétrico do PLDA, não do calibrador).
- ≈ ECE máximo essencialmente igual — diferença abaixo do limiar visual nos plots. A previsão de melhora de 0,015 bits em §7.1 é otimista; em prática a diferença é menor.

Recomendação operacional para Projeto 1 - PLDA: se for usar Bi-Gaussianized, prefira `KDE_SMOOTHED` — produz output esteticamente correto sem custo numérico. Se busca mínimo absoluto de ECE, LogReg continua sendo a escolha (ver documento separado de comparação).

8. Caveats e limitações

8.1 Departure do paper

KDE_SMOOTHED é uma variante deste projeto, não está no paper Morrison 2024. Se você publicar resultados usando KDE_SMOOTHED, deve declará-lo explicitamente como modificação. Sugestão de linguagem:

> *"Calibration was performed using a kernel-smoothed variant of Bi-Gaussianized calibration (Morrison 2024). The empirical piecewise-linear CDF G(x) of §2.7 was replaced by an analytically-integrated Gaussian KDE estimate, G(x) = 0.5·F̃_g(x) + 0.5·F̃_i(x), with per-class bandwidths determined by Silverman's rule of thumb. The remainder of the algorithm (steps 1-4 and 6) follows Morrison (2024) exactly. The modification is motivated by stairstep artifacts observed in calibrated log-LR distributions when N_gen in calibration training is sparse; the kernel-smoothed estimate eliminates these artifacts without altering the EER or the target bi-Gaussian system σ."*

8.2 Bandwidth bias

Silverman's rule of thumb assume distribuição aproximadamente Gaussiana. Para distribuições **multimodais por classe** (raro em scores forenses, mas possível com hard-mining agressivo), a regra sobre-suaviza modas finas. Mitigação: usar bandwidth manual menor, ou aceitar como propriedade do método (afinal o biGauss assume bi-gaussiano alvo, então modas finas seriam perdidas de qualquer forma).

8.3 Cauda longa não-Gaussiana

Se uma classe tem cauda extrema (caso PLDA TWO_COVARIANCE, impostores extending to -200), Silverman pode escolher bandwidth grande demais devido ao std inflado pela cauda. A implementação usa min(std, IQR/1.349) exatamente para mitigar isso — IQR/1.349 é robusto contra outliers. Em dúvida, inspeccione o bandwidth efetivo: print(model.metadata) mostra os h_g e h_i usados.

8.4 Não corrige assimetria de support

Como observado em §4.2: se os scores brutos têm supportes muito diferentes por classe (caso PLDA L2), o Tippett plot da saída calibrada continuará assimétrico, independentemente de EMPIRICAL_LINEAR ou KDE_SMOOTHED. Essa assimetria é uma propriedade dos dados brutos e seria corrigida (parcialmente) apenas com transformação prévia dos scores (whitening, clip, arcsinh, etc.). KDE_SMOOTHED resolve discretização, não asimetria.

9. Compatibilidade com Morrison (2024)

Esta seção resume **o que muda** e **o que não muda** em relação ao paper canônico, para facilitar revisão científica.

Componente	Paper Morrison §	Em KDE_SMOOTHED
Passo 1: Score data + labels	§2.2 step 1	Idêntico
Passo 2: Calibrador inicial (LogReg/KDE)	§2.2 step 2	Idêntico (LOGREG / KDE / NONE configurável)
Passo 3: Cllr ou EER do output do passo 2	§2.2 step 3 + §2.3	Idêntico (portado localmente sem acoplar a outros serviços)
Passo 4: $\sigma^2 \leftrightarrow$ Cllr ou EER	§2.3 (Eq. 8) e §2.5 (Eq. 10)	Idêntico (ambas equações implementadas)
Passo 5: Construção de G(x)	§2.7 (piecewise linear)	**SUBSTITUÍDO** por KDE-smoothed quando gx_method=KDE_SMOOTHED
Passo 5: Pesos 0.5/(N_c+1) por classe	§2.7	**PRESERVADO** ($G = 0.5 \cdot \hat{F}_g + 0.5 \cdot \hat{F}_i$)
Passo 5: Non-extrapolation	§2.7 (clamp em x_min, x_max)	**PRESERVADO** via gx_clamp_to_training_range=True
Passo 5: Pesos somam < 1 ($G(x_{max}) < 1$)	§2.7 (evita F^{-1} infinito)	**PRESERVADO** via rescale leve
Passo 6: $F^{-1}(u)$ da mistura alvo	§2.7 (Eq. 11)	Idêntico (INTERP1D ou BRENT_ROOT)
Sistema alvo: $0.5 \cdot N(\pm\sigma^2/2, \sigma^2)$	§2.1 (Eq. 4)	Idêntico

9.1 Argumento de equivalência assintótica

Quando $N \to \infty$, ambos os métodos convergem para a mesma G(x): a CDF verdadeira da população dos scores. Especificamente:

- **EMPIRICAL_LINEAR** converge por Glivenko-Cantelli: a CDF empírica converge uniformemente para a CDF verdadeira.
- **KDE_SMOOTHED** converge se o bandwidth $h \to 0$ com $N \cdot h \to \infty$ (condição satisfeita por Silverman: $h \propto N^{(-1/5)}$).
- Para N finito, os dois métodos diferem por O(h) em norma sup. Em treinos grandes (validado N=20k: max diff 0.07 no miolo), a diferença é negligível para fins práticos.

9.2 Por que esta variante não está no paper

Morrison (2024) discute KDE como **calibrador inicial** (§2.6, §3.6) e mostra que essa variante é a melhor entre EER, LogReg, KDE, PAV pelos resultados Monte Carlo. Porém, o passo 5 (construção de G(x)) é sempre piecewise-linear no paper.

Não há discussão explícita no paper sobre suavização de G(x). Os datasets usados pelo paper têm N_gen entre 100 (sintéticos §3.2/3.3) e 320 (vidro §3.5). Nesses regimes, os stairsteps existem mas são **menos visíveis** porque os Tippett plots usam escala log□□-cumulativa (que comprime os steps) e as KDE plots dos log-LR são suavizadas pelo próprio KDE de plotagem (bandwidth do *display*, não do modelo).

No FaceAudit, plotamos a distribuição calibrada via KDE Gaussiana com bandwidth Silverman *dos próprios outputs* — bandwidth pequena porque há ~4.000 outputs. Isso torna os stairsteps muito mais visíveis que nos plots do paper original. KDE_SMOOTHED é uma resposta natural a essa diferença de

apresentação.

Documento gerado automaticamente pelo script docs/_build_bigauss_kde_smoothed_pdf.py. Para regenerar com gráficos atualizados: python docs/_build_bigauss_kde_smoothed_pdf.py. Implementação em faceaudit/models/calibration/bi_gaussianized_calibration_model.py.